

Прикладная теория типов

Домашнее задание 3 (λ -исчисления, соответствие Карри-Ховарда)

23 ноября 2025 г.

Домашняя работа принимается до 23:59 12 декабря 2025, кроме задач, помеченных звёздочкой, которые принимаются до конца семестра. Решения "теоретических" задач можно набрать в TeX или написать разборчивым текстом на бумаге и отсканировать. **Формат сдачи зависит от задачи:** базовые задачи сдаются в виде pdf и ml файлов, большие программные задачи (ML-pipeline, GADT, unsoundness в системе типов Rust) — в виде ссылки на репозиторий (GitHub/GitLab) или архива с кодом и отдельного pdf с объяснениями (детали указаны в условиях задач). Всё сдаётся на почту m.voronov@gse.cs.msu.ru. Вопросы по домашнему заданию можно задавать или по почте, или в ТГ-группе курса.

- (1 балл) Сколько существует $\lambda 2$ контекстов для следующего списка деклараций: $\alpha : \star, \beta : \star, \gamma : \star, f : \alpha \rightarrow \beta, g : \gamma \rightarrow \beta, x : \beta$. Приведите обоснование.
- Для каждого предтерма определите, типизуется ли он $\lambda 2$, если да - найдите его тип а-ля Карри и а-ля Чёрч, если нет - обоснуйте, почему предтерм не типизируем:
 - (2 балла) $\lambda x.(xx)(xx)$
 - (2 балла) $S \equiv \lambda xyz.xz(yz)$
- Используя соответствие Карри-Ховарда и OCaml, докажите следующие логические утверждения пропозициональной логики. При реализации используйте произведение (`'a * 'b`) для $\wedge$ и сумму (например, `type ('a,'b) either = Left of 'a | Right of 'b`) для $\vee$.
 - (2 балла) $A \vee B \implies B \vee A$
 - (2 балла) $(A \wedge B) \implies (D \vee C) \implies (D \implies F) \implies (C \implies G) \implies ((A \wedge F) \vee (B \wedge G))$
 - (2 балла) $((C \implies A) \wedge (C \implies B)) \implies (C \implies (A \wedge B))$
- Используя соответствие Карри-Ховарда и OCaml, покажите следующие изоморфизмы типов (под $a \sim b$ понимается наличие прямой и обратной функций, которые являются взаимными обратными). Для каждого изоморфизма: (а) переведите в типы OCaml, (б) реализуйте обе функции с явными типами, (с) объясните, почему они взаимно обратны.
 - (3 балла) $a^{b+c} \sim a^b \times a^c$
 - (3 балла) $(a^b)^c \sim a^{b \times c}$
 - (2 балла) $a^0 \sim 1$
 - (2 балла) $a^1 \sim a$

- Рассмотрим конструктор типов:

$$\text{Flip} \equiv \lambda F : \star \rightarrow \star \rightarrow \star. \lambda \alpha : \star. \lambda \beta : \star. F \beta \alpha$$

- (1 балл) Определите вид конструктора Flip
 - (1 балл) Постройте полное дерево вывода вида в системе $\lambda\omega$ с использованием правил из лекции 9
 - (1 балл) Приведите пример применения: чему равен Flip Either Int Bool? Укажите вид результата.
- Рассмотрим следующие определения в System F_ω :
 - Functor $F \equiv \forall \alpha \beta. (\alpha \rightarrow \beta) \rightarrow F \alpha \rightarrow F \beta$
 - `fmapList : Functor List` — реализация для списков
 - `fmapMaybe : Functor Maybe` — реализация для Maybe
 - (1 балл) Какой вид должен иметь параметр F в определении Functor?

- (2 балла) Докажите, что если F и G — функторы, то $\text{Compose } F \, G$ — тоже функтор. Приведите определение $\text{fmap}_{\text{Compose}}$ через fmap_F и fmap_G .
- (1 балл) Объясните, почему в Java/C# нельзя выразить такую абстракцию, а в Haskell можно. Какого механизмы не хватает языкам.

7. Используя систему λP и зависимые типы:

- (1 балл) Спроектируйте и формализуйте тип функции `replicate`, которая создаёт вектор длины n из n копий элемента. Используйте Π -типы и тип $\text{Vec } A \, n$ (вектор элементов типа A длины n).
- (1 балл) Спроектируйте и формализуйте тип функции `safeHead`, которая извлекает первый элемент *непустого* вектора. Как типы гарантируют, что вектор точно непустой?
- (1 балл) Объясните, почему попытка вызвать `safeHead` на пустом векторе `nil` приведёт к ошибке типизации.
- (1 балл) Приведите пример корректного вызова `safeHead` с явным указанием всех аргументов и их типов.

8. Рассмотрим Σ -типы в системе λP :

- (1 балл) Объясните разницу между типами $A \times \text{Proof}$ и $\Sigma x : A. P(x)$ на конкретном примере.
- (1 балл) Спроектируйте тип функции `filter`, которая фильтрует список по предикату и возвращает новый список. Проблема: длина результата заранее неизвестна. Как использовать Σ -тип, чтобы возвращаемое значение включало и сам результат, и его длину?
- (1 балл) Формализуйте утверждение «существует чётное число больше 2» как Σ -тип.
- (1 балл) Постройте обитатель этого типа (т.е. докажите утверждение).

9. Объясните простыми словами, понятными семилетнему ребёнку:

- (3 балла) Что такое виды (kinds) и зачем они нужны. Почему мы не можем написать `List Maybe`, но можем написать `List Int` и `Compose List Maybe`? Приведите бытовую аналогию (например, с коробками, функциями, рецептами и т.п.).
- (3 балла) Что такое зависимые типы на примере $\text{Vec } A \, n$ (вектор длины n). Чем `Vec Int 3` отличается от обычного `List Int`? Почему компилятор не даст нам вызвать `head` на пустом векторе, но позволит на непустом? Приведите аналогию из повседневной жизни.

10. **Типобезопасный ML-pipeline: от базовых гарантий до защиты от adversarial attacks.**

Требуется спроектировать систему типов для безопасной библиотеки машинного обучения, которая гарантирует корректность и устойчивость к атакам на уровне типов:

- **Базовые гарантии:** предотвращение shape mismatch, использования необученных моделей, data leakage;
- **Безопасность данных:** разделение adversarial и clean данных, защита от data poisoning;
- **Privacy:** отслеживание применения differential privacy и sensitive data.

Контекст: В production ML-системах критически важно предотвратить не только технические ошибки, но и атаки: adversarial examples, data poisoning, model inversion, membership inference. Типы — один из вариантов этой защиты.

Язык: код для решения этой задачи рекомендуется писать на любом языке, который поддерживает ADT/GADT, помимо этого Вы можете использовать Ваш любимый язык в предположении, что в нём есть ADT/GADT, т.е. по сути псевдокод, основанный на каком-то языке, а также можете напрямую использовать любой понятный псевдокод.

Замечание о структуре: Подпункты 1–6 представляют собой независимые аспекты безопасности ML-систем. Каждый можно решать отдельно, используя одну технику (phantom types и/или GADT). Подпункт 7 требует систематического анализа всех реализованных инвариантов. Опционально: попробуйте объединить несколько модулей (например, dataset с privacy и lifecycle модели) в единую систему типов — это усилит Ваше решение и может дать дополнительные баллы.

Оценивание задания: Это задание участвует в субъективном конкурсе, как и задание с объяснением теорем из ДЗ1. Будет оцениваться полнота реализации, объяснений и креатива: за первое место можно получить +50% баллов, за второе — +30%, за третье — +20% от общего числа баллов.

Формат сдачи: Сдайте ссылку на публичный репозиторий (GitHub/GitLab) с кодом и README, либо архив с исходниками. Обязательно приложите отдельный PDF-файл с объяснениями дизайна системы типов, примерами использования и анализом инвариантов. Альтернативно: можно включить все объяснения в виде подробных комментариев прямо в коде, если они достаточно развёрнутые.

- (4 балла) **Shape-safe архитектура.** Спроектируйте систему типов для слоёв нейронной сети, которая гарантирует согласованность размерностей на этапе компиляции.

Требования:

- Тензоры должны быть параметризованы размерностями (например, batch size и число features)
- Слои нейросети (Dense, ReLU, Dropout) должны иметь типы, отражающие их входную и выходную размерность
- Композиция слоёв должна быть типобезопасной: компилятор должен отвергать попытку соединить слой с выходом 128 и слой с входом 256
- Используйте phantom types и/или GADT

Что нужно сделать:

- Определите типы для тензоров и слоёв
- Опишите типы для 2-3 конкретных слоёв (Dense, ReLU, и др.)
- Определите функцию композиции слоёв с корректными типами
- Приведите пример некорректной архитектуры, которая будет отвергнута компилятором
- Объясните, как phantom types и/или GADT обеспечивают проверку согласованности размерностей

- (3 балла) **Lifecycle модели и состояния.** Спроектируйте систему типов для отслеживания состояния модели машинного обучения.

Требования:

- Модель должна проходить через состояния: Untrained → Trained → Validated
- Нельзя делать предсказания на необученной модели
- Нельзя повторно обучить уже обученную модель (без явного сброса)
- Валидация возможна только после обучения
- Используйте phantom types и/или GADT для кодирования состояний

Что нужно сделать:

- Определите типы для модели с состояниями (например, ('inp', 'out', 'state') model)
- Спроектируйте типы функций init, train, validate, predict так, чтобы они гарантировали корректный lifecycle
- Приведите 2-3 примера некорректных вызовов, которые будут отвергнуты компилятором
- Объясните, какие инварианты безопасности закодированы в типах

- (3 балла) **Типобезопасные датасеты и защита от data leakage.** Спроектируйте систему типов для отслеживания разбиения данных и их обработки.

Требования:

- Датасеты должны различаться по split'y: Train, Validation, Test
- Данные могут быть Raw (сырые) или Normalized (обработанные)
- Нельзя обучить модель на Test-данных
- Нельзя подать необработанные (Raw) данные в функцию, требующую Normalized
- Нормализация параметров должна вычисляться только на Train-данных

Что нужно сделать:

- Спроектируйте параметризованный тип для датасетов (используя phantom types)
- Определите типы функций: загрузка данных, нормализация, разбиение на train/test, обучение модели

- Покажите, как типы гарантируют, что:
 - * модель обучается только на Train-данных
 - * в обучение подаются только Normalized данные
 - * Test-данные не попадают в обучение
- Приведите примеры некорректного кода, который отвергнет компилятор
- (2 балла) **Связь с λP и зависимыми типами.** Выберите *один* из инвариантов из предыдущих подпунктов (например, «нельзя вызывать `predict` на необученной модели» или «модель обучается только на Train-данных») и:
 - Запишите его как формулу в стиле λP (используя Π - и, при необходимости, Σ -типы).
 - Объясните, как ваш phantom-type дизайн в ML-языке приближённо реализует этот инвариант, и почему без полноценных зависимых типов (как в λP) он не может быть выражен *ровно*.
- (3 балла) **Защита от adversarial examples.** Adversarial examples — это специально сгенерированные входы, которые обманывают модель. В production системах критично не смешивать adversarial данные с чистыми при обучении.

Требования:

- Данные должны быть помечены как Clean, Adversarial или Suspicious
- Обычная функция `train` не должна принимать Adversarial данные
- Adversarial training должен быть отдельной операцией, принимающей и clean и adversarial примеры
- Adversarial примеры должны генерироваться только из clean данных с помощью модели
- Функция `evaluation` должна работать с данными любого происхождения

Что нужно сделать:

- Спроектируйте параметризованный тип для данных с меткой происхождения
- Определите типы функций: загрузка clean данных, генерация adversarial примеров, adversarial training, evaluation
- Объясните, как типы предотвращают случайное смешивание adversarial и clean данных при обычном обучении
- Приведите пример реальной атаки (FGSM, PGD, DeepFool) и объясните, как ваш дизайн типов помогает безопасно работать с такими данными
- (Бонус) Как расширить систему для защиты от data poisoning (вредоносных меток)?
- (4 балла) **Privacy-aware типы.** Differential privacy — методика добавления шума в данные/модель для защиты приватности пользователей.

Требования:

- Данные должны быть помечены как Public или Private с параметром приватности ϵ
- Нельзя опубликовать (release) приватные данные без достаточного уровня защиты
- Система должна отслеживать privacy budget при композиции операций
- Тип должен отражать уровень приватности

Что нужно сделать:

- Спроектируйте параметризованный тип для данных с уровнем приватности
- Определите типы функций для добавления шума и публикации данных
- Объясните, как типы могут отслеживать privacy budget (композицию операций с разными ϵ)
- Обсудите, почему это близко к зависимым типам (тип зависит от значения ϵ)
- Приведите пример из реальности: Google RAPPOR, Apple differential privacy, или federated learning
- (2 балла) **Систематический анализ инвариантов безопасности.** На основе вашей реализации из пунктов 1-6:
 - Составьте полный список инвариантов безопасности, которые гарантируются вашими типами (минимум 5-6 инвариантов). Для каждого укажите: что запрещается, какой тип это обеспечивает, какую ошибку даст компилятор.

- Обсудите ограничения: какие важные инварианты НЕ выразимы в вашей системе типов и требуют runtime проверок, тестов или зависимых типов? Например:

- * "Accuracy модели не меньше baseline"
- * "Dataset сбалансирован (одинаковое число примеров каждого класса)"
- * "Нет дубликатов между train и test"

11. (6 баллов)* Покажите, что следующие термы не типизируемы в $System F$, где I, K - комбинаторы:

- $(\lambda sz.s(sz))(\lambda sz.s(sz))K$
- $(\lambda zy.y(zI)(zK))(\lambda x.xx)$

Контекст и мотивация:

Одним из важнейших свойств $System F$ является *сильная нормализация* (strong normalization): каждый типизируемый терм обязательно редуцируется к нормальной форме за конечное число шагов, то есть вычисление всегда завершается. Это означает, что в $System F$ нельзя выразить бесконечные циклы или рекурсивные функции (для этого требуются дополнительные конструкции, такие как фиксированная точка).

Из сильной нормализации следует важное наблюдение: **если терм типизируем в $System F$, то он нормализуем**. Однако обратное неверно: существуют термы, которые нормализуются (завершают вычисление), но при этом не типизируются в $System F$. Это показывает, что система типов $System F$ является *неполной*: она отвергает некоторые «безопасные» программы.

Приведённые выше термы являются именно такими примерами: оба терма редуцируются к нормальной форме (проверьте это!), но не имеют типа в $System F$. Ваша задача — формально доказать их нетипизируемость, используя свойства системы типов.

12. * **Параметричность и теоремы бесплатно (Theorems for Free).**

В $System F$ параметричность полиморфных функций позволяет автоматически выводить нетривиальные теоремы о их поведении исключительно из типовых сигнатур, без знания реализации.

- (2 балла) Докажите, что для любой функции $f : \forall \alpha. \alpha \rightarrow \alpha$ в $System F$ верно $f = \lambda x.x$ (с точностью до $\beta\eta$ -эквивалентности). Используйте рассуждение о параметричности.
- (3 балла) Рассмотрим функцию $map : \forall \alpha \beta. (\alpha \rightarrow \beta) \rightarrow List \alpha \rightarrow List \beta$. Докажите **бесплатную теорему**: для любых функций $f : A \rightarrow B$ и $g : B \rightarrow C$ выполняется:

$$map(g \circ f) = map g \circ map f$$

Используйте только параметричность, не зная конкретной реализации map .

- (3 балла) Рассмотрим тип $\forall \alpha. (\alpha \rightarrow \alpha) \rightarrow \alpha \rightarrow \alpha$. Докажите с помощью параметричности, что любой замкнутый терм этого типа эквивалентен некоторому числу Чёрча n (функции «применить f к x ровно n раз»). Опишите общий вид всех таких термов в форме $\Lambda \alpha. \lambda f^{\alpha \rightarrow \alpha}. \lambda x^{\alpha}. \dots$ и объясните связь с кодированием натуральных чисел Чёрча.

13. * **Когерентность (coherence) и неоднозначность в системах type classes.**

В $System F$ с type classes когерентность означает, что для данного типа и класса типов существует **не более одной** реализации (instance). Это свойство критически важно для предсказуемости поведения программ и оптимизаций компилятора.

- (2 балла) Рассмотрим класс типов Monoid:

$$Monoid \alpha \equiv \{empty : \alpha, mappend : \alpha \rightarrow \alpha \rightarrow \alpha\}$$

Приведите пример типа, для которого существует несколько математически корректных, но различных инстансов Monoid. Почему наличие нескольких возможных инстансов нарушает когерентность? Какие проблемы это создаёт для компилятора и программиста?

- (2 балла) Объясните проблему с orphan instances (сиротскими инстансами): почему разрешение определять инстансы классов типов в модулях, не содержащих ни определения класса, ни определения типа, нарушает когерентность системы? Приведите конкретный пример ситуации, когда два различных модуля определяют orphan instances для одной и той же пары (класс, тип), и объясните, какой конфликт возникает при их одновременном импорте.
- (2 балла) Рассмотрим overlapping instances (перекрывающиеся инстансы) в Haskell:

```
instance Show a => Show [a]           -- (1) общий случай
instance Show [Char]                -- (2) специальный случай
```

При вызове `show "hello"` (где тип `"hello"` это `[Char]`) какой инстанс должен быть выбран? Опишите стратегию разрешения конфликтов в Haskell (принцип `most specific instance`). Почему GHC требует явного разрешения через pragma `OverlappingInstances` или `OVERLAPPING`? Как это связано с `trait coherence` в Rust?

14. * GADT для type-safe DSL.

Введение: ADT vs GADT. На лекциях мы изучали алгебраические типы данных (ADT), которые представляют собой суммы произведений:

```
type expr =                                (* обычный ADT *)
  | Lit of int
  | Add of expr * expr
  | If of expr * expr * expr
```

Проблема: такой тип позволяет построить некорректные выражения вроде `Add (Lit 5, If (...))`, и ошибка обнаружится только при интерпретации. **Generalized Algebraic Data Types (GADT)** расширяют ADT, позволяя разным конструкторам возвращать разные инстанции параметризованного типа:

```
type _ expr =                              (* GADT с phantom type *)
  | Lit : int -> int expr
  | Add : int expr * int expr -> int expr
  | If  : bool expr * 'a expr * 'a expr -> 'a expr
```

Теперь `Add (Lit 5, If (...))` — type error на этапе компиляции! Phantom type `'a` в `'a expr` отслеживает тип значения выражения.

Формат сдачи: Сдайте ссылку на публичный репозиторий (GitHub/GitLab) с кодом, либо архив с исходниками и отдельный PDF с объяснениями. Код должен компилироваться и содержать примеры использования. Альтернативно: можно включить все объяснения в виде подробных комментариев прямо в коде.

- (2 балла) **Понимание разницы ADT vs GADT.** Рассмотрим обычный ADT:

```
type expr =                                (* обычный ADT *)
  | IntLit of int
  | BoolLit of bool
  | Add of expr * expr
  | If of expr * expr * expr

val eval : expr -> ???                     (* какой тип результата? *)
```

Объясните:

- Почему функция `eval` не может возвращать «просто `int`» или «просто `bool`» в зависимости от конкретного выражения, а вынужден всегда возвращать один и тот же суммарный тип (`variant`)

- Какие проверки нужно делать в runtime (например, в Add проверять, что оба аргумента — числа)
- Как GADT позволяет сделать тип результата зависящим от структуры выражения за счёт phantom type parameter
- (3 балла) Спроектируйте расширенный GADT для типобезопасных выражений:

```
type _ expr =
  | Lit      : int -> int expr
  | String   : string -> string expr
  | Add      : int expr -> int expr -> int expr
  | Concat   : string expr -> string expr -> string expr
  | Eq       : 'a expr -> 'a expr -> bool expr
  | If       : bool expr -> 'a expr -> 'a expr -> 'a expr
  | Length   : string expr -> int expr
```

Для каждого из следующих выражений определите, корректно ли оно типизируется, и если нет — какую ошибку даст компилятор:

```
– Add (Lit 5) (String "hello")
– Concat (String "a") (String "b")
– If (Eq (Lit 5) (Lit 10)) (String "yes") (String "no")
– If (Lit 42) (Lit 1) (Lit 2)
– Length (If (Eq (Lit 1) (Lit 1)) (String "a") (String "b"))
```

- (2 балла) Реализуйте type-safe interpreter без динамических проверок типов:

```
val eval : 'a expr -> 'a
```

Покажите примеры корректных и некорректных (отвергнутых компилятором) выражений. Объясните, как phantom type parameter 'a гарантирует type safety.

- (1 балл) Приведите реальные примеры использования GADT в индустрии:

- SQL builders: Diesel (Rust), Opaleye (Haskell), Selda (Haskell)
- Parser combinators: парсеры с typed AST
- Staged compilation: MetaOCaml, BER MetaOCaml
- Domain-specific languages: конфигурационные языки, query languages

Объясните связь GADT с зависимыми типами: как GADT эмулируют некоторые возможности λP в System F_ω .

15. * Формальный анализ нарушения soundness в системе типов компилятора Rust

Требуется провести исследование дефекта в системе типов компилятора Rust, приводящего к нарушению свойства soundness (корректности). Под нарушением soundness понимается ситуация, когда система типов допускает компиляцию программы, нарушающей инварианты безопасности памяти или типов, что противоречит теоретическим гарантиям типизированных языков программирования.

Цель работы: исследовать взаимосвязь между теоретическими основами систем типов (полиморфизм, изоморфизм Карри-Ховарда, soundness, coherence, леммы progress и preservation) и их практической реализацией в промышленных компиляторах, а также проанализировать последствия нарушения теоретических свойств для безопасности программного обеспечения.

Критерий оценки: за это задание также можно получить до +200% дополнительных баллов, если анализ будет субъективно исчерпывающим, Вы реально выберете сложный недостаток, напишите подробную статью в свой блог/habr/medium/... и за любой креатив помимо того, что требуется в самом задании. Критерий выставления дополнительных баллов - субъективный, но Вы можете связаться с лектором и уточнить, как и что может быть оценено.

Критерии выбора дефекта

Необходимо выбрать один дефект из официального репозитория компилятора Rust ([rust-lang/rust](https://github.com/rust-lang/rust)), удовлетворяющий следующим требованиям:

- Дефект классифицирован метками **T-types** (относящийся к подсистеме типов) и **C-bug** (подтверждённый дефект)
- Дефект находится в открытом состоянии либо был закрыт не ранее чем за 12 месяцев до момента сдачи работы
- Дефект приводит к нарушению soundness системы типов, формально определяемому как возможность получения undefined behavior без использования конструкций языка, помеченных ключевым словом **unsafe**

Методика поиска: рекомендуется использовать систему отслеживания дефектов GitHub Issues с фильтром `is:issue state:open label:C-bug label:T-types`. Для уточнения поиска рекомендуется добавить метку `label:I-unsound`, явно маркирующую дефекты, нарушающие soundness. Допускается выбор недавно закрытых дефектов при недостаточном количестве открытых.

Замечание: При наличии неопределённости относительно соответствия выбранного дефекта требованиям задачи или его сложности допускается предварительное согласование выбора с преподавателем.

Формат сдачи: Сдайте PDF-документ с формальным анализом дефекта. Для дополнительной части (эксплуатация дефекта) приложите ссылку на Rust Playground, ссылку на репозиторий GitHub/GitLab с исходным кодом .rs файла, либо сам .rs файл. Весь код должен быть снабжён детальными комментариями, объясняющими каждый этап эксплуатации.

Часть 1: Формальный анализ дефекта

Анализ должен включать следующие обязательные компоненты:

- **Формальная спецификация дефекта:**
 - Идентификатор дефекта в системе GitHub Issues с полной ссылкой
 - Формализованное описание проблемы с использованием терминологии теории типов
 - Версия компилятора, в которой дефект воспроизводится (если установлено)
 - Минимальный воспроизводимый пример (minimal reproducible example, MRE) — корректно компилирующаяся программа, демонстрирующая дефект
- **Технический анализ причин дефекта:**
 - Идентификация компонента системы типов, содержащего дефект (trait resolution, lifetime inference, variance checking, borrow checker, coherence, specialization, и т.д.)
 - Формальное объяснение причины некорректного поведения компилятора: отсутствующая либо некорректная проверка типизации
 - Установление связи с теоретическими концепциями: определение нарушенных свойств системы типов (progress, preservation, parametricity, coherence)
 - Анализ предложенных в обсуждении дефекта гипотез о его причинах и возможных методах устранения (при наличии)
- **Анализ практических последствий:**
 - Классификация нарушенных гарантий безопасности (memory safety, thread safety, type safety)
 - Перечисление возможных проявлений undefined behavior (use-after-free, data races, type confusion, dangling references, invalid values)
 - Оценка вероятности случайного проявления дефекта в производственном коде
 - Конструирование конкретного примера программы, в которой дефект приводит к undefined behavior или нарушению инвариантов безопасности
- **Теоретико-типовой анализ:**
 - Формализация дефекта в терминах System F, System F_ω или зависимых типов
 - При наличии связи с trait-системой: описание в терминах type classes либо экзистенциальных типов
 - Анализ соотношения дефекта с фундаментальными леммами progress и preservation для типизированных исчислений

Важно: Работы, представляющие собой пересказ содержания issue без самостоятельного анализа и формализации, будут оцениваться на минимальном уровне. Требуется демонстрация понимания теоретических основ и способности применять формальные методы анализа.

Часть 2: Эксплуатация дефекта в изолированной среде

Опционально можно продемонстрировать методы эксплуатации дефекта в контексте изолированной среды исполнения (sandbox), в которой программа может исполнять произвольный Rust-код (например, Rust Playground, система плагинов), но запрещено использование конструкций `unsafe`, Foreign Function Interface (FFI) и системных вызовов для доступа к файловой системе или сети.

- **Формальное описание метода эксплуатации:** детализированное объяснение методологии использования нарушения soundness для получения возможности исполнения произвольного кода (*arbitrary code execution*) или доступа к произвольным областям памяти. Должно включать формализацию этапов эксплуатации и теоретическое обоснование корректности метода.
- **Работающая реализация эксплуатации (Proof of Concept):** программа, корректно компилирующаяся стабильной версией компилятора Rust без использования `unsafe`, демонстрирующая одно из следующих проявлений undefined behavior:
 - Чтение или модификация произвольных областей памяти
 - Вызов произвольной функции посредством некорректного указателя `vtable`
 - Конструирование значения типа с нарушенными инвариантами (например, значение типа `bool`, отличное от 0 и 1)
 - Нарушение правил алиасинга (aliasing), приводящее к состоянию гонки (data race) в однопоточной программе

Замечание: Требование полноценной демонстрации удалённого исполнения кода (remote code execution) не предъявляется. Достаточно убедительной демонстрации проявления undefined behavior в ограниченном сценарии.

Классификация типов дефектов soundness (допускается выбор дефекта любой категории):

- *Нарушения когерентности trait-системы (trait coherence violations):* конфликтующие реализации (`impl`) для одного типа, нарушающие свойство единственности (`uniqueness`)
- *Дефекты вывода времён жизни (lifetime soundness):* некорректный алгоритм вывода времён жизни (lifetime inference), допускающий создание висящих ссылок (dangling references)
- *Нарушения правил вариантности (variance issues):* некорректное определение ковариантности/контравариантности обобщённых параметров типов
- *Дефекты специализации (specialization bugs):* ошибки в механизме перекрывающихся реализаций при активированной возможности `min_specialization`
- *Дефекты ограничений высшего ранга (HRTB — Higher-Rank Trait Bounds):* некорректная обработка ограничений вида `for<'a>`
- *Дефекты обобщения по константам (const generics):* нарушения soundness в типах, параметризованных константами
- *Дефекты обобщённых ассоциированных типов (GATs — Generic Associated Types):* ошибки в реализации недавно стабилизированного механизма
- *Дефекты проверки корректности деструкторов (drop check):* некорректные проверки безопасности исполнения деструкторов

Требования к оформлению

- **Часть 1:** PDF-документ, содержащий формальный анализ (допускается объединение с решениями других задач в единый документ). Рекомендуемый объём: 1-2 страницы для базового уровня, 2-3 страниц для глубокого анализа.
- **Часть 2 (при наличии PoC):** исходный код на языке Rust (файл с расширением `.rs`) либо постоянная ссылка на Rust Playground, сопровождаемые детальными комментариями, формализующими каждый этап эксплуатации.

Рекомендуемая структура документа:

- (a) Введение: формулировка проблемы и обоснование её значимости для теории и практики типизированных языков
- (b) Спецификация дефекта: MRE с формальным описанием некорректного поведения компилятора
- (c) Технический анализ: идентификация нарушенного компонента системы типов и причин некорректного поведения
- (d) Теоретический анализ: формализация в терминах теории типов (soundness, progress, preservation, System F/ F_ω)
- (e) Анализ практических последствий: классификация возможных проявлений undefined behavior
- (f) Заключение: обсуждение возможных методов устранения дефекта, выводы
- (g) Приложение (опционально): демонстрация эксплуатации с формальными комментариями

Критерии оценивания

Часть 1 (максимум 15 баллов):

- **8–10 баллов** (удовлетворительный уровень): предоставлен MRE с описанием дефекта, поверхностное объяснение проблемы, минимальное использование формального аппарата теории типов. Работа носит преимущественно описательный характер без глубокого анализа.
- **11–13 баллов** (хороший уровень): детальное техническое объяснение причин некорректного поведения компилятора, систематический анализ практических последствий, явное установление связей с фундаментальными концепциями (soundness, progress, preservation). Наличие самостоятельно разработанных примеров и иллюстраций.
- **14–15 баллов** (отличный уровень): всё вышеперечисленное плюс глубокий формальный анализ в терминах теории типов (System F, System F_ω , type classes, экзистенциальные типы), самостоятельные наблюдения относительно природы дефекта, обсуждение возможных методов устранения с формальным обоснованием. Демонстрация понимания нарушенных инвариантов на формальном уровне.

Часть 2 (максимум 10 баллов):

- **4 балла:** формальное описание метода эксплуатации с детализацией этапов
- **6 баллов:** работающая реализация эксплуатации, демонстрирующая undefined behavior без использования `unsafe`
- **Максимальная оценка: 25 баллов** (15 за часть 1 + 10 за часть 2)

Рекомендуемая литература

- *The Rustonomicon*. [Официальная документация](#) — формальное описание семантики `unsafe` Rust и инвариантов безопасности
- *The Rust Reference*. [«Subtyping and Variance»](#) — формальная спецификация правил вариантности в системе типов Rust
- *GitHub Issues*: [метка I-unsound](#) — реестр известных дефектов, нарушающих soundness
- *Counterexamples in Type Systems*. [counterexamples.org](#) — сборник soundness bugs в различных языках программирования с объяснениями и примерами
- Jung R. et al. [«RustBelt: Securing the Foundations of the Rust Programming Language»](#), POPL 2018 — формальная верификация безопасности Rust в системе Coq
- Matsakis N. [«Baby Steps»](#) — технический блог о теоретических основах дизайна системы типов Rust